

EE/CS 148B HW 1

Build an LM from Scratch*

Lead TAs: Vansh Tibrewal, Damiano Marsili

Spring 2026

1 Assignment Overview

In this assignment, you will build all the components needed to train a standard Transformer language model (LM) from scratch and train an LM on the TinyStories dataset.

What you will implement

1. Byte-pair encoding (BPE) tokenizer (§2)
2. Transformer language model (LM) (§3)
3. The cross-entropy loss function (§4)
4. The training loop putting it all together (§5)

What you will run

1. Train a BPE tokenizer on the TinyStories dataset.
2. Run your trained tokenizer on the dataset to convert it into a sequence of integer IDs.
3. Train a Transformer LM on the TinyStories dataset.
4. Generate samples and evaluate perplexity using the trained Transformer LM.

What you can use We expect you to build these components from scratch. In particular, for sections 2, 3, 4 you may not use any definitions from `torch.nn` or `torch.nn.functional` except:

- `torch.nn.Parameter`
- Container classes in `torch.nn` (e.g., `Module`, `ModuleList`, `Sequential`, etc.)¹

You may use other PyTorch functionality where explicitly permitted below, especially in section 5. When in doubt, consider if using it compromises the “from-scratch” ethos of the assignment and feel free to ask for clarification on Piazza.

*This assignment is adapted from HW1 of Stanford’s CS336 course by Percy Liang and Tatsunori Hashimoto: <https://github.com/stanford-cs336/assignment1-basics>. We adapt the assignment for this course.

¹See [PyTorch.org/docs/stable/nn.html#containers](https://pytorch.org/docs/stable/nn.html#containers) for a full list.

What the code looks like All the assignment code is available on GitHub at:
github.com/caltech-eecs148b/hw1

1. `eeecs148b_hw1/*`: This is where you write your code. Note that there’s no code in here—you can do whatever you want from scratch!
2. `adapters.py`: This is a set of functionality that your code must have. For each piece of functionality (e.g., transformer block), implement it (e.g., `run_transformer_block`) by simply invoking your code. Note: your changes to `adapters.py` should not contain any substantive logic; this is glue code.
3. `test_*.py`: This contains unit tests for each part of the assignment you will implement (e.g., `test_transformer_block`). You may use these to validate your implementation, or you may write your own. These tests will invoke the hooks defined in `adapters.py`. Don’t edit the test files.

Where to get the data This assignment will use the pre-processed dataset TinyStories [1]. The dataset is a single, large plaintext file. To download it, see the `README.md` in the repo.

How to submit You will submit the following files to Gradescope:

- `writeup.pdf`: Answer all the written questions. Please typeset your responses.
- `code.zip`: Contains all the code you’ve written. Create a **private** repository on GitHub and push your code to it. Submit this GitHub repo by following the directions on Gradescope.

2 Byte-Pair Encoding (BPE) Tokenizer

In the first part of the assignment, we will train and implement a byte-level byte-pair encoding (BPE) tokenizer [3, 4]. In particular, we will represent arbitrary (Unicode) strings as a sequence of bytes and train our BPE tokenizer on this byte sequence. Later, we will use this tokenizer to encode text (a string) into tokens (a sequence of integers) for language modeling.

2.1 The Unicode Standard

Unicode is a text encoding standard that maps characters to integer code points. As of Unicode 16.0 (released in September 2024), the standard defines 154,998 characters across 168 scripts. For example, the character “x” has the code point 120. In Python, you can use the `ord()` function to convert a single Unicode character into its integer representation. The `chr()` function converts an integer Unicode code point into a string with the corresponding character.

```
>>> ord('x')
120
>>> chr(120)
'x'
```

2.2 Unicode Encodings

While the Unicode standard defines a mapping from characters to code points (integers), it's impractical to train tokenizers directly on Unicode codepoints, since the vocabulary would be prohibitively large (around 150K items) and sparse (since many characters are quite rare). Instead, we'll use a Unicode encoding (UTF-8), which converts a Unicode character into a sequence of bytes.

To encode a Unicode string into UTF-8, we can use the `encode()` function in Python. To access the underlying byte values for a Python `bytes` object, we can iterate over it (e.g., call `list()`). Finally, we can use the `decode()` function to decode a UTF-8 byte string into a Unicode string.

By converting our Unicode codepoints into a sequence of bytes (e.g., via the UTF-8 encoding), we are essentially taking a sequence of codepoints (integers in the range 0 to 154,997) and transforming it into a sequence of byte values (integers in the range 0 to 255). The 256-length byte vocabulary is much more manageable to deal with. When using byte-level tokenization, we do not need to worry about out-of-vocabulary tokens, since we know that any input text can be expressed as a sequence of integers from 0 to 255.

2.3 Subword Tokenization

While byte-level tokenization can alleviate the out-of-vocabulary issues faced by word-level tokenizers, tokenizing text into bytes results in extremely long input sequences. This slows down model training, since a sentence with 10 words might only be 10 tokens long in a word-level language model, but could be 50 or more tokens long in a character-level model (depending on the length of the words). Processing these longer sequences requires more computation at each step of the model. Furthermore, language modeling on byte sequences is difficult because the longer input sequences create long-term dependencies in the data.

Subword tokenization is a midpoint between word-level tokenizers and byte-level tokenizers. Note that a byte-level tokenizer's vocabulary has 256 entries (byte values are 0 to 255). A subword tokenizer trades-off a larger vocabulary size for better compression of the input byte sequence. For example, if the byte sequence `b'the'` often occurs in our raw text training data, assigning it an entry in the vocabulary would reduce this 3-token sequence to a single token.

How do we select these subword units to add to our vocabulary? Sennrich et al. [3] propose to use byte-pair encoding (BPE; [5]), a compression algorithm that iteratively replaces ("merges") the most frequent pair of bytes with a single, new unused index. Note that this algorithm adds subword tokens to our vocabulary to maximize the compression of our input sequences—if a word occurs in our input text enough times, it'll be represented as a single subword unit.

Subword tokenizers with vocabularies constructed via BPE are often called BPE tokenizers. In this assignment, we'll implement a byte-level BPE tokenizer, where the vocabulary items are bytes or merged sequences of bytes, which give us the best of both worlds in terms of out-of-vocabulary handling and manageable input sequence lengths. The process of constructing the BPE tokenizer vocabulary is known as "training" the BPE tokenizer.

2.4 BPE Tokenizer Training

The BPE tokenizer training procedure consists of three main steps.

Vocabulary initialization The tokenizer vocabulary is a one-to-one mapping from bytestring token to integer ID. Since we're training a byte-level BPE tokenizer, our initial vocabulary is simply the set of all bytes. Since there are 256 possible byte values, our initial vocabulary is of size 256.

Pre-tokenization Once you have a vocabulary, you could, in principle, count how often bytes occur next to each other in your text and begin merging them starting with the most frequent pair of bytes. However, this is quite computationally expensive, since we’d have to go take a full pass over the corpus each time we merge.

Insight (BPE): Naive merging

Naively merging pairs of bytes next to each other in text can also lead to undesirable tokenization of similar concepts. Why might this be an issue?

Deliverable: A 1-2 sentence explanation of why this is an issue. **hint:** consider “dog!” and “dog?” both appearing in the same corpus.

In addition, directly merging bytes across the corpus may result in tokens that differ only in punctuation (e.g., `dog!` vs. `dog.`). These tokens would get completely different token IDs, even though they are likely to have high semantic similarity (since they differ only in punctuation).

To avoid this, we pre-tokenize the corpus. You can think of this as a coarse-grained tokenization over the corpus that helps us count how often pairs of characters appear. For example, the word ‘`text`’ might be a pre-token that appears 10 times. In this case, when we count how often the characters ‘`t`’ and ‘`e`’ appear next to each other, we will see that the word ‘`text`’ has ‘`t`’ and ‘`e`’ adjacent and we can increment their count by 10 instead of looking through the corpus. Since we’re training a byte-level BPE model, each pre-token is represented as a sequence of UTF-8 bytes.

We’ll use a regex-based pre-tokenizer (used by GPT-2; [6]) from github.com/openai/tiktoken/pull/234/files:

```
>>> PAT = r'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+\s+(?!\S)|\s+'''
```

When using it in your code, you should use `re.finditer` to avoid storing the pre-tokenized words as you construct your mapping from pre-tokens to their counts.

Compute BPE merges Now that we’ve converted our input text into pre-tokens and represented each pre-token as a sequence of UTF-8 bytes, we can compute the BPE merges (i.e., train the BPE tokenizer). At a high level, the BPE algorithm iteratively counts every pair of bytes and identifies the pair with the highest frequency (“A”, “B”). Every occurrence of this most frequent pair (“A”, “B”) is then merged, i.e., replaced with a new token “AB”. This new merged token is added to our vocabulary; as a result, the final vocabulary after BPE training is the size of the initial vocabulary (256 in our case), plus the number of BPE merge operations performed during training. For efficiency during BPE training, we do not consider pairs that cross pre-token boundaries.² When computing merges, deterministically break ties in pair frequency by preferring the lexicographically greater pair. For example, if the pairs (“A”, “B”), (“A”, “C”), (“B”, “ZZ”), and (“BA”, “A”) all have the highest frequency, we’d merge (“BA”, “A”):

```
>>> max([("A", "B"), ("A", "C"), ("B", "ZZ"), ("BA", "A")])
('BA', 'A')
```

²Note that the original BPE formulation [3] specifies the inclusion of an end-of-word token. We do not add an end-of-word-token when training byte-level BPE models because all bytes (including whitespace and punctuation) are included in the model’s vocabulary. Since we’re explicitly representing spaces and punctuation, the learned BPE merges will naturally reflect these word boundaries.

Special tokens Some strings (e.g., `<|endoftext|>`) are used to encode metadata (e.g., boundaries between documents). When encoding text, it's often desirable to treat some strings as “special tokens” that should never be split into multiple tokens (i.e., will always be preserved as a single token). For example, the end-of-sequence string `<|endoftext|>` should always be preserved as a single token (i.e., a single integer ID), so we know when to stop generating from the language model. These special tokens must be added to the vocabulary, so they have a corresponding fixed token ID.

2.5 Experimenting with BPE Tokenizer Training

Let's train a byte-level BPE tokenizer on the TinyStories dataset. Instructions to find / download the dataset can be found in Section 1. Before you start, we recommend taking a look at the TinyStories dataset to get a sense of what's in the data.

Removing special tokens before pre-tokenization Before running pre-tokenization with the regex pattern (using `re.finditer`), you should strip out all special tokens from your corpus. Make sure that you split on your special tokens, so that no merging can occur across the text they delimit. For example, if you have a corpus (or chunk) like `[Doc 1]<|endoftext|>[Doc 2]`, you should split on the special token `<|endoftext|>`, and pre-tokenize `[Doc 1]` and `[Doc 2]` separately, so that no merging can occur across the document boundary. This can be done using `re.split` with `"|".join(special_tokens)` as the delimiter (with careful use of `re.escape` since `|` may occur in the special tokens). The test `test_train_bpe_special_tokens` will test for this.

Optimizing the merging step The naïve implementation of BPE training is slow because for every merge, it iterates over all byte pairs to identify the most frequent pair. However, the only pair counts that change after each merge are those that overlap with the merged pair. Thus, BPE training speed can be improved by indexing the counts of all pairs and incrementally updating these counts, rather than explicitly iterating over each pair of bytes to count pair frequencies. You can get significant speedups with this caching procedure.

Problem (train_bpe): BPE Tokenizer Training

Deliverable: Write a function that, given a path to an input text file, trains a (byte-level) BPE tokenizer. Your BPE training function should handle (at least) the following input parameters:

input_path: `str` Path to a text file with BPE tokenizer training data.

vocab_size: `int` A positive integer that defines the maximum final vocabulary size (including the initial byte vocabulary, vocabulary items produced from merging, and any special tokens).

special_tokens: `list[str]` A list of strings to add to the vocabulary. These special tokens do not otherwise affect BPE training.

Your BPE training function should return the resulting vocabulary and merges:

vocab: `dict[int, bytes]` The tokenizer vocabulary, a mapping from `int` (token ID in the vocabulary) to `bytes` (token bytes).

merges: `list[tuple[bytes, bytes]]` A list of BPE merges produced from training. Each list item is a tuple of bytes (`<token1>`, `<token2>`), representing that `<token1>` was merged with `<token2>`. The merges should be ordered by order of creation.

To test your BPE training function against our provided tests, you will first need to implement the test adapter at `[adapters.run_train_bpe]`. Then, run `uv run pytest tests/test_train_bpe.py`. Your implementation should be able to pass all tests.

Problem (`train_bpe_tinystories`): BPE Training on TinyStories

Train a byte-level BPE tokenizer on the TinyStories dataset, using a maximum vocabulary size of 10,000. Make sure to add the TinyStories `<|endoftext|>` special token to the vocabulary. Serialize the resulting vocabulary and merges to disk for further inspection. What is the longest token in the vocabulary? Does it make sense?

Resource requirements: ≤ 30 minutes (no GPUs), ≤ 30 GB RAM

2.6 BPE Tokenizer: Encoding and Decoding

In the previous part of the assignment, we implemented a function to train a BPE tokenizer on input text to obtain a tokenizer vocabulary and a list of BPE merges. Now, we will implement a BPE tokenizer that loads a provided vocabulary and list of merges and uses them to encode and decode text to/from token IDs.

2.6.1 Encoding text

The process of encoding text by BPE mirrors how we train the BPE vocabulary. There are a few major steps.

Step 1: Pre-tokenize. We first pre-tokenize the sequence and represent each pre-token as a sequence of UTF-8 bytes, just as we did in BPE training. We will be merging these bytes within each pre-token into vocabulary elements, handling each pre-token independently (no merges across pre-token boundaries).

Step 2: Apply the merges. We then take the sequence of vocabulary element merges created during BPE training, and apply it to our pre-tokens in the same order of creation.

Example (`bpe_encoding`): BPE encoding example

For example, suppose our input string is `'the cat ate'`, our vocabulary is `{0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't', 6: b'th', 7: b' c', 8: b' a', 9: b'the', 10: b' at'}`, and our learned merges are `[(b't', b'h'), (b' ', b'c'), (b' ', b'a'), (b'th', b'e'), (b' a', b't')]`. First, our pre-tokenizer would split this string into `['the', ' cat', ' ate']`.

Then, we'll look at each pre-token and apply the BPE merges.

The first pre-token `'the'` is initially represented as `[b't', b'h', b'e']`. Looking at our list of merges, we identify the first applicable merge to be `(b't', b'h')`, and use that to transform the pre-token into `[b'th', b'e']`. Then, we go back to the list of merges and identify the next applicable merge to be `(b'th', b'e')`, which transforms the pre-token into `[b'the']`. Finally, looking back at the list of merges, we see that there are no more

that apply to the string (since the entire pre-token has been merged into a single token), so we are done applying the BPE merges. The corresponding integer sequence is [9]. Repeating this process for the remaining pre-tokens, we see that the pre-token ‘ cat’ is represented as [b‘ c’, b‘a’, b‘t’] after applying the BPE merges, which becomes the integer sequence [7, 1, 5]. The final pre-token ‘ ate’ is [b‘ at’, b‘e’] after applying the BPE merges, which becomes the integer sequence [10, 3]. Thus, the final result of encoding our input string is [9, 7, 1, 5, 10, 3].

Special tokens. Your tokenizer should be able to properly handle user-defined special tokens when encoding text (provided when constructing the tokenizer).

2.6.2 Decoding text

To decode a sequence of integer token IDs back to raw text, we can simply look up each ID’s corresponding entries in the vocabulary (a byte sequence), concatenate them together, and then decode the bytes to a Unicode string. Note that input IDs are not guaranteed to map to valid Unicode strings (since a user could input any sequence of integer IDs). In the case that the input token IDs do not produce a valid Unicode string, you should replace the malformed bytes with the official Unicode replacement character U+FFFD.³ The `errors` argument of `bytes.decode` controls how Unicode decoding errors are handled, and using `errors='replace'` will automatically replace malformed data with the replacement marker.

Problem (tokenizer): Implementing the tokenizer

Deliverable: Implement a `Tokenizer` class that, given a vocabulary and a list of merges, encodes text into integer IDs and decodes integer IDs into text. Your tokenizer should also support user-provided special tokens (appending them to the vocabulary if they aren’t already there). We recommend the following interface:

```
def __init__(self, vocab, merges, special_tokens=None) Construct a tokenizer from
    a given vocabulary, list of merges, and (optionally) a list of special tokens. This function
    should accept the following parameters:
```

- `vocab`: `dict[int, bytes]`
- `merges`: `list[tuple[bytes, bytes]]`
- `special_tokens`: `list[str] | None = None`

```
def from_files(cls, vocab_filepath, merges_filepath, special_tokens=None)
    Class method that constructs and returns a Tokenizer from a serialized vocabulary
    and list of merges (in the same format that your BPE training code output) and
    (optionally) a list of special tokens. This method should accept the following additional
    parameters:
```

- `vocab_filepath`: `str`
- `merges_filepath`: `str`

³See [en.wikipedia.org/wiki/Specials_\(Unicode_block\)#Replacement_character](https://en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character) for more information about the Unicode replacement character.

```

    • special.tokens: list[str] | None = None

def encode(self, text: str) -> list[int] Encode an input text into a sequence of
    token IDs.

def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int] Given an
    iterable of strings (e.g., a Python file handle), return a generator that lazily yields
    token IDs. This supports tokenizing large inputs without materializing the full token
    sequence in memory.

def decode(self, ids: list[int]) -> str Decode a sequence of token IDs into text.

```

To test your `Tokenizer` against our provided tests, you will first need to implement the test adapter at `[adapters.get_tokenizer]`. Then, run `uv run pytest tests/test_tokenizer.py`. Your implementation should be able to pass all tests.

2.7 Experiments

Problem (tokenizer_experiments): Experiments with tokenizers

- Sample 10 documents from TinyStories. Using your previously-trained TinyStories tokenizer, encode these sampled documents into integer IDs. What is the tokenizer's compression ratio (bytes/token)?
- Using your TinyStories tokenizer, encode the respective training and development datasets into a sequence of integer token IDs. We'll use this later to train our language model. We recommend serializing the token IDs as a NumPy array of datatype `uint16`. Why is `uint16` an appropriate choice?

3 Transformer Language Model Architecture

3.1 Basic Building Blocks: Linear and Embedding Modules

3.1.1 Parameter Initialization

Training neural networks effectively often requires careful initialization of the model parameters—bad initializations can lead to undesirable behavior such as vanishing or exploding gradients. Pre-norm transformers are unusually robust to initializations, but they can still have a significant impact on training speed and convergence. Here are some approximate initializations that should work well for most cases, which you should use for this assignment:

- **Linear weights:** $\mathcal{N}(\mu = 0, \sigma^2 = \frac{2}{d_{\text{in}} + d_{\text{out}}})$ truncated at $[-3\sigma, 3\sigma]$.
- **Embedding:** $\mathcal{N}(\mu = 0, \sigma^2 = 1)$ truncated at $[-3, 3]$.
- **LayerNorm:** weight initialized to 1 and bias initialized to 0.

You should use `torch.nn.init.trunc_normal_` to initialize the truncated normal weights.

3.1.2 Linear Module

Linear layers are a fundamental building block of Transformers and neural nets in general. First, you will implement your own `Linear` class that inherits from `torch.nn.Module` and performs a linear transformation:

$$y = Wx. \tag{1}$$

Note that we do not include a bias term, following most modern LLMs.

Problem (linear): Implementing the linear module

Deliverable: Implement a `Linear` class that inherits from `torch.nn.Module` and performs a linear transformation. Your implementation should follow the interface of PyTorch’s built-in `nn.Linear` module, except for not having a bias argument or parameter. We recommend the following interface:

```
def __init__(self, in_features, out_features, device=None, dtype=None)
    Construct a linear transformation module.
```

```
def forward(self, x: torch.Tensor) -> torch.Tensor
    Apply the linear transformation to the input.
```

Make sure to:

- subclass `nn.Module`
- call the superclass constructor
- construct and store your parameter as W (not W^T) for memory ordering reasons, putting it in an `nn.Parameter`
- of course, don’t use `nn.Linear` or `nn.functional.linear`

For initializations, use the settings from above along with `torch.nn.init.trunc_normal_` to initialize the weights.

To test your `Linear` module, implement the test adapter at `[adapters.run_linear]`. The adapter should load the given weights into your `Linear` module. You can use `Module.load_state_dict` for this purpose. Then, run `uv run pytest -k test_linear`.

3.1.3 Embedding Module

The first layer of the Transformer is an embedding layer that maps integer token IDs into a vector space of dimension `d_model`. We will implement a custom `Embedding` class that inherits from `torch.nn.Module` (so you should not use `nn.Embedding`). The forward method should select the embedding vector for each token ID by indexing into an embedding matrix of shape `(vocab_size, d_model)` using a `torch.LongTensor` of token IDs with shape `(batch_size, sequence_length)`.

Problem (embedding): Implement the embedding module

Deliverable: Implement the `Embedding` class that inherits from `torch.nn.Module` and performs an embedding lookup. Your implementation should follow the interface of PyTorch’s

built-in `nn.Embedding` module. We recommend the following interface:

```
def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None)
    Construct an embedding module.
```

```
def forward(self, token_ids: torch.Tensor) -> torch.Tensor
    Lookup the embedding vectors for the given token IDs.
```

Make sure to:

- subclass `nn.Module`
- call the superclass constructor
- initialize your embedding matrix as a `nn.Parameter`
- store the embedding matrix with the `d_model` being the final dimension
- of course, don't use `nn.Embedding` or `nn.functional.embedding`

Again, use the settings from above for initialization, and use `torch.nn.init.trunc_normal_` to initialize the weights.

To test your implementation, implement the test adapter at `[adapters.run_embedding]`. Then, run `uv run pytest -k test_embedding`.

3.2 Pre-Norm Transformer Block

Each Transformer block has two sub-layers: a multi-head self-attention mechanism and a position-wise feed-forward network [8].

In the original Transformer paper, the model uses a residual connection around each of the two sub-layers, followed by layer normalization. This architecture is commonly known as the “post-norm” Transformer, since layer normalization is applied to the sublayer output. However, a variety of work has found that moving layer normalization from the output of each sub-layer to the input of each sub-layer (with an additional layer normalization after the final Transformer block) improves Transformer training stability [9, 10]. The output of each Transformer block sub-layer is then added to the sub-layer input via the residual connection [8]. An intuition for pre-norm is that there is a clean “residual stream” without any normalization going from the input embeddings to the final output of the Transformer, which is purported to improve gradient flow. This pre-norm Transformer is now the standard used in language models today (e.g., GPT-3, LLaMA, PaLM, etc.), so we will implement this variant. We will walk through each of the components of a pre-norm Transformer block, implementing them in sequence.

3.2.1 Layer Normalization

In this assignment, we will use the standard LayerNorm from the original Transformer implementation of Vaswani et al. [8]. Given a vector $a \in \mathbb{R}^{d_{\text{model}}}$ of activations, LayerNorm first normalizes each activation using the mean and variance of the full vector, then applies a learned affine transformation:

$$\text{LayerNorm}(a_i) = \frac{a_i - \mu(a)}{\sqrt{\sigma^2(a) + \varepsilon}} g_i + b_i, \quad (2)$$

where

$$\mu(a) = \frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} a_i \quad \text{and} \quad \sigma^2(a) = \frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} (a_i - \mu(a))^2.$$

Here, g_i and b_i are learnable parameters (there are d_{model} of each), and ε is a hyperparameter that is often fixed at $1\text{e-}5$.

You should upcast your input to `torch.float32` to prevent numerical issues during the mean and variance computations. Overall, your forward method should look like:

```
in_dtype = x.dtype
x = x.to(torch.float32)
# Your code here performing LayerNorm
...
result = ...
# Return the result in the original dtype
return result.to(in_dtype)
```

Problem (layernorm): Implement Layer Normalization

Deliverable: Implement `LayerNorm` as a `torch.nn.Module`. We recommend the following interface:

```
def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)
    Construct the LayerNorm module.
```

```
def forward(self, x: torch.Tensor) -> torch.Tensor
    Process an input tensor of shape (batch_size, sequence_length, d_model) and return a tensor of the same shape.
```

Note: Remember to upcast your input to `torch.float32` before performing the normalization (and later downcast to the original dtype), as described above.

To test your implementation, implement the test adapter at `[adapters.run_layernorm]`. Then, run `uv run pytest -k test_layernorm`.

3.2.2 Position-Wise Feed-Forward Network

In the original Transformer paper (section 3.3 of Vaswani et al. [8]), the Transformer feed-forward network consists of two linear transformations with a ReLU activation between them:

$$\text{ReLU}(x) = \max(0, x). \quad (3)$$

The dimensionality of the inner feed-forward layer is typically $4 \times d_{\text{model}}$.

In this assignment, we will use this standard 2-layer position-wise feed-forward network. Given an input vector $x \in \mathbb{R}^{d_{\text{model}}}$, the feed-forward network is:

$$\text{FFN}(x) = W_2(\text{ReLU}(W_1x)), \quad (4)$$

where $W_1 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$, $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, and typically $d_{\text{ff}} = 4d_{\text{model}}$.

As in the rest of this assignment, we omit bias terms in the linear layers.

Problem (positionwise_feedforward): Implement the position-wise feed-forward network

Deliverable: Implement the standard 2-layer Transformer feed-forward network with a ReLU activation between the two linear transformations.

Note: Do not use `torch.relu` or `torch.nn.functional.relu`; implement the ReLU activation yourself. You should also use your own `Linear` module for the two linear transformations rather than PyTorch’s built-in `nn.Linear`.

Use $d_{\text{ff}} = 4 \times d_{\text{model}}$ in your implementation. To test your implementation, implement the test adapter at `[adapters.run_positionwise_feedforward]`. Then, run `uv run pytest -k test_positionwise_feedforward`.

3.2.3 Sinusoidal Positional Embeddings

To inject positional information into the model, we will use fixed sinusoidal positional embeddings, as introduced in the original Transformer paper of Vaswani et al. [8]. For a token at position p and embedding dimension i , the positional embedding is defined by

$$\text{PE}(p, 2i) = \sin\left(\frac{p}{10000^{2i/d_{\text{model}}}}\right), \quad \text{PE}(p, 2i + 1) = \cos\left(\frac{p}{10000^{2i/d_{\text{model}}}}\right). \quad (5)$$

Thus, each position is assigned a fixed vector in $\mathbb{R}^{d_{\text{model}}}$ whose even coordinates are sine values and whose odd coordinates are cosine values at different frequencies.

Unlike learned positional embeddings, these embeddings have no learnable parameters. A good implementation should precompute a tensor of shape `(max_seq_len, d_model)` containing the positional embeddings and store it as a non-persistent buffer using `self.register_buffer(persistent=False)`. At runtime, given token positions, your module should simply index into this precomputed tensor.

Problem (sinusoidal_positional_embedding): Implement sinusoidal positional embeddings

Deliverable: Implement a class `SinusoidalPositionalEncoding` that returns sinusoidal positional embeddings for the requested token positions. We recommend the following interface:

```
def __init__(self, d_model: int, max_seq_len: int, device=None, dtype=None)
    Construct the sinusoidal positional embedding module and precompute the positional embedding buffer.

def forward(self, token_positions: torch.Tensor) -> torch.Tensor
    Given token positions of shape (..., seq_len), return positional embeddings of shape (..., seq_len, d_model).
```

Note: You may use `torch.sin`, `torch.cos`, `torch.exp`, and `torch.arange` in this implementation. Since the embeddings are fixed rather than learned, they should be stored as a buffer, not as an `nn.Parameter`. You may assume that `d_model` is even.

To test your implementation, complete `[adapters.run_sinusoidal_positional_embedding]` and make sure it passes `uv run pytest -k test_sinusoidal_positional_embedding`.

3.2.4 Scaled Dot-Product Attention

We will now implement scaled dot-product attention as described in Vaswani et al. [8] (section 3.2.1). As a preliminary step, the definition of the Attention operation will make use of softmax, an operation that takes an unnormalized vector of scores and turns it into a normalized distribution:

$$\text{softmax}(v)_i = \frac{\exp(v_i)}{\sum_{j=1}^n \exp(v_j)}. \quad (6)$$

Insight (softmax): Handling numerical instability

In the definition of softmax above, you may notice that $\exp(v_i)$ can become `inf` for large values. Then, `inf/inf = NaN`. How can we avoid this numerical issue in our implementation without changing what the softmax operation does?

Deliverable: A 1-2 sentence explanation of how to avoid this issue.

Problem (softmax): Implement softmax

Deliverable: Write a function to apply the softmax operation on a tensor. Your function should take two parameters: a tensor and a dimension i , and apply softmax to the i -th dimension of the input tensor. The output tensor should have the same shape as the input tensor, but its i -th dimension will now have a normalized probability distribution.

To test your implementation, complete `[adapters.run_softmax]` and make sure it passes `uv run pytest -k test_softmax_matches_pytorch`.

We can now define the Attention operation mathematically as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (7)$$

where $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{m \times d_k}$, and $V \in \mathbb{R}^{m \times d_v}$. Here, Q , K and V are all inputs to this operation—note that these are not the learnable parameters.

Masking: It is sometimes convenient to mask the output of an attention operation. A mask should have the shape $M \in \{\text{True}, \text{False}\}^{n \times m}$, and each row i of this boolean matrix indicates which keys the query i should attend to. Canonically (and slightly confusingly), a value of `True` at position (i, j) indicates that the query i does attend to the key j , and a value of `False` indicates that the query does not attend to the key. In other words, “information flows” at (i, j) pairs with value `True`. For example, consider a 1×3 mask matrix with entries `[[True, True, False]]`. The single query vector attends only to the first two keys. Computationally, it will be much more efficient to use masking than to compute attention on subsequences.

Insight: Masking

What mathematical operation can we apply to the pre-softmax values $\left(\frac{QK^T}{\sqrt{d_k}}\right)$ to mask any entry where the mask matrix is `False`?

Deliverable: A 1 sentence solution.

Problem (scaled_dot_product_attention): Implement scaled dot-product attention

Deliverable: Implement the scaled dot-product attention function. Your implementation should handle keys and queries of shape `(batch_size, ..., seq_len, d_k)` and values of shape `(batch_size, ..., seq_len, d_v)`, where `...` represents any number of other batch-like dimensions (if provided). The implementation should return an output with the shape `(batch_size, ..., seq_len, d_v)`.

Your implementation should also support an optional user-provided boolean mask of shape `(seq_len, seq_len)`. The attention probabilities of positions with a mask value of `True` should collectively sum to 1, and the attention probabilities of positions with a mask value of `False` should be zero.

To test your implementation against our provided tests, you will need to implement the test adapter at `[adapters.run_scaled_dot_product_attention]`.

`uv run pytest -k test_scaled_dot_product_attention` tests your implementation on third-order input tensors, while `uv run pytest -k test_4d_scaled_dot_product_attention` tests your implementation on fourth-order input tensors.

3.2.5 Causal Multi-Head Self-Attention

We will implement multi-head self-attention as described in section 3.2.2 of Vaswani et al. [8]. Recall that, mathematically, the operation of applying multi-head attention is defined as follows:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \quad (8)$$

$$\text{for head}_i = \text{Attention}(Q_i, K_i, V_i) \quad (9)$$

with Q_i, K_i, V_i being slice number $i \in \{1, \dots, h\}$ of size d_k or d_v of the embedding dimension for Q , K , and V respectively. With Attention being the scaled dot-product attention operation defined in §3.5.4. From this we can form the multi-head self-attention operation:

$$\text{MultiHeadSelfAttention}(x) = W^O \text{MultiHead}(W^Q x, W^K x, W^V x) \quad (10)$$

Here, the learnable parameters are $W^Q \in \mathbb{R}^{hd_k \times d_{\text{model}}}$, $W^K \in \mathbb{R}^{hd_k \times d_{\text{model}}}$, $W^V \in \mathbb{R}^{hd_v \times d_{\text{model}}}$, and $W^O \in \mathbb{R}^{d_{\text{model}} \times hd_v}$. Since the Q s, K s, and V s are sliced in the multi-head attention operation, we can think of W^Q , W^K and W^V as being separated for each head along the output dimension. When you have this working, you should be computing the key, value, and query projections in a total of three matrix multiplies.⁴

Causal masking. Your implementation should prevent the model from attending to future tokens in the sequence. In other words, if the model is given a token sequence $t_1, \dots, t_n$, and we want to calculate the next-word predictions for the prefix $t_1, \dots, t_i$ (where $i < n$), the model should not be able to access (attend to) the token representations at positions $t_{i+1}, \dots, t_n$ since it will not have access to these tokens when generating text during inference (and these future tokens leak information about the identity of the true next word, trivializing the language modeling pre-training objective). For an input token sequence $t_1, \dots, t_n$ we can naively prevent access to future

⁴As a stretch goal, try combining the key, query, and value projections into a single weight matrix so you only need a single matrix multiply.

tokens by running multi-head self-attention n times (for the n unique prefixes in the sequence). Instead, we'll use causal attention masking, which allows token i to attend to all positions $j \leq i$ in the sequence. You can use `torch.triu` or a broadcasted index comparison to construct this mask, and you should take advantage of the fact that your scaled dot-product attention implementation already supports attention masking.

Problem (multihead_self_attention): Implement causal multi-head self-attention

Deliverable: Implement causal multi-head self-attention as a `torch.nn.Module`. Your implementation should accept (at least) the following parameters:

`d_model: int` Dimensionality of the Transformer block inputs.

`num_heads: int` Number of heads to use in multi-head self-attention.

Following Vaswani et al. [8], set $d_k = d_v = d_{\text{model}}/h$. To test your implementation against our provided tests, implement the test adapter at `[adapters.run_multihead_self_attention]`. Then, run `uv run pytest -k test_multihead_self_attention` to test your implementation.

3.3 The Full Transformer LM

Let's begin by assembling the Transformer block. A Transformer block contains two 'sublayers', one for the multihead self attention, and another for the feed-forward network. In each sublayer, we first perform LayerNorm, then the main operation (MHA/FF), finally adding in the residual connection.

To be concrete, the first half (the first 'sub-layer') of the Transformer block should be implementing the following set of updates to produce an output y from an input x ,

$$y = x + \text{MultiHeadSelfAttention}(\text{LayerNorm}(x)). \quad (11)$$

Problem (transformer_block): Implement the Transformer block

Implement the pre-norm Transformer block as described in §3.2. Your Transformer block should accept (at least) the following parameters.

`d_model: int` Dimensionality of the Transformer block inputs.

`num_heads: int` Number of heads to use in multi-head self-attention.

`d_ff: int` Dimensionality of the position-wise feed-forward inner layer.

To test your implementation, implement the adapter `[adapters.run_transformer_block]`. Then run `uv run pytest -k test_transformer_block` to test your implementation.

Deliverable: Transformer block code that passes the provided tests.

Now we put the blocks together. Use the embedding module you implemented to map token IDs to vectors, add sinusoidal positional embeddings, feed the result through `num_layers` Transformer blocks, apply a final LayerNorm, and then pass the result through an LM head to obtain logits over the vocabulary.

Problem (transformer_lm): Implementing the Transformer LM

Time to put it all together! Implement the Transformer language model. At minimum, your implementation should accept all the aforementioned construction parameters for the Transformer block, as well as these additional parameters:

vocab_size: `int` The size of the vocabulary, necessary for determining the dimensionality of the token embedding matrix and the LM head output.

context_length: `int` The maximum context length, necessary for determining the size of the sinusoidal positional embedding table.

num_layers: `int` The number of Transformer blocks to use.

Your implementation should combine token embeddings, sinusoidal positional embeddings, stacked pre-norm Transformer blocks, a final LayerNorm, and an LM head that maps hidden states to vocabulary logits. To test your implementation against our provided tests, you will first need to implement the test adapter at `[adapters.run_transformer_lm]`. Then, run `uv run pytest -k test_transformer_lm` to test your implementation.

Deliverable: A Transformer LM module that passes the above tests.

4 Training a Transformer LM

We now have the steps to preprocess the data (via tokenizer) and the model (Transformer). What remains is to build all of the code to support training. This consists of the following:

- **Loss:** we need to define the loss function (cross-entropy).
- **Training loop:** we need all the supporting infrastructure that loads data, saves checkpoints, and manages training.

4.1 Cross-entropy loss

Recall that the Transformer language model defines a distribution $p_\theta(x_{i+1}|x_{1:i})$ for each sequence x of length $m+1$ and $i = 1, \dots, m$. Given a training set $\mathcal{D}$ consisting of sequences of length m , we define the standard cross-entropy (negative log-likelihood) loss function:

$$\ell(\theta; \mathcal{D}) = \frac{1}{|\mathcal{D}|m} \sum_{x \in \mathcal{D}} \sum_{i=1}^m -\log p_\theta(x_{i+1}|x_{1:i}). \quad (12)$$

(Note that a single forward pass in the Transformer yields $p_\theta(x_{i+1}|x_{1:i})$ for all $i = 1, \dots, m$.)

In particular, the Transformer computes logits $o_i \in \mathbb{R}^{\text{vocab_size}}$ for each position i , which results in:⁵

$$p(x_{i+1}|x_{1:i}) = \text{softmax}(o_i)[x_{i+1}] = \frac{\exp(o_i[x_{i+1}])}{\sum_{a=1}^{\text{vocab_size}} \exp(o_i[a])}. \quad (13)$$

The cross entropy loss is generally defined with respect to the vector of logits $o_i \in \mathbb{R}^{\text{vocab_size}}$ and target x_{i+1} .⁶

⁵Note that $o_i[k]$ refers to value at index k of the vector o_i .

⁶This corresponds to the cross entropy between the Dirac delta distribution over x_{i+1} and the predicted $\text{softmax}(o_i)$ distribution.

Implementing the cross entropy loss requires some care with numerical issues, just like in the case of softmax.

Problem (cross_entropy): Implement Cross entropy

Deliverable: Write a function to compute the cross entropy loss, which takes in predicted logits (o_i) and targets (x_{i+1}) and computes the cross entropy $\ell_i = -\log \text{softmax}(o_i)[x_{i+1}]$. Your function should handle the following:

- Subtract the largest element for numerical stability.
- Cancel out log and exp whenever possible.
- Handle any additional batch dimensions and return the average across the batch. As with section 3.3, we assume batch-like dimensions always come first, before the vocabulary size dimension.

Implement `[adapters.run_cross_entropy]`, then run `uv run pytest -k test_cross_entropy` to test your implementation.

Perplexity Cross entropy suffices for training, but when we evaluate the model, we also want to report perplexity. For a sequence of length m where we suffer cross-entropy losses $\ell_1, \dots, \ell_m$:

$$\text{perplexity} = \exp \left(\frac{1}{m} \sum_{i=1}^m \ell_i \right). \quad (14)$$

Insight: Perplexity

Why might perplexity be a better metric for evaluating LLMs?

Deliverable: A 1-2 sentence explanation.

5 Training loop

5.1 Data Loader

The tokenized data (e.g., that you prepared in `tokenizer_experiments`) is a single sequence of tokens $x = (x_1, \dots, x_n)$. Even though the source data might consist of separate documents (e.g., different web pages, or source code files), a common practice is to concatenate all of those into a single sequence of tokens, adding a delimiter between them (such as the `<|endoftext|>` token).

A data loader turns this into a stream of batches, where each batch consists of B sequences of length m , paired with the corresponding next tokens, also with length m . For example, for $B = 1$, $m = 3$, $([x_2, x_3, x_4], [x_3, x_4, x_5])$ would be one potential batch.

Loading data in this way simplifies training for a number of reasons. First, any $1 \leq i < n - m$ gives a valid training sequence, so sampling sequences are trivial. Since all training sequences have the same length, there's no need to pad input sequences, which improves hardware utilization (also by increasing batch size B). Finally, we also don't need to fully load the full dataset to sample training data, making it easy to handle large datasets that might not otherwise fit in memory.

Problem (data_loading): Implement data loading

Deliverable: Write a function that takes a numpy array x (integer array with token IDs), a `batch_size`, a `context_length` and a PyTorch device string (e.g., `'cpu'` or `'cuda:0'`), and returns a pair of tensors: the sampled input sequences and the corresponding next-token targets. Both tensors should have shape `(batch_size, context_length)` containing token IDs, and both should be placed on the requested device. To test your implementation against our provided tests, you will first need to implement the test adapter at `[adapters.run_get_batch]`. Then, run `uv run pytest -k test_get_batch` to test your implementation.

What if the dataset is too big to load into memory? We can use a Unix systemcall named `mmap` which maps a file on disk to virtual memory, and lazily loads the file contents when that memory location is accessed. Thus, you can “pretend” you have the entire dataset in memory. Numpy implements this through `np.memmap` (or the flag `mmap_mode='r'` to `np.load`, if you originally saved the array with `np.save`), which will return a numpy array-like object that loads the entries on-demand as you access them. When sampling from your dataset (i.e., a numpy array) during training, be sure load the dataset in memory-mapped mode (via `np.memmap` or the flag `mmap_mode='r'` to `np.load`, depending on how you saved the array). Make sure you also specify a dtype that matches the array that you’re loading. It may be helpful to explicitly verify that the memory-mapped data looks correct (e.g., doesn’t contain values beyond the expected vocabulary size).

5.2 Training loop

Now, it’s finally time to put all of the components you implemented together into your main training script. It will pay off to make it easy to start training runs with different hyperparameters (e.g., by taking them as command-line arguments), since you will be doing these many times later to study how different choices impact training.

Problem (training_together): Put it together

Deliverable: Write a script that runs a training loop to train your model on user-provided input. In particular, we recommend that your training script allow for (at least) the following:

- Ability to configure and control the various model and optimizer hyperparameters.
- Memory-efficient loading of training and validation large datasets with `np.memmap`.
- Saving checkpoints to a user-provided path.
- Periodically logging training and validation performance (e.g., to console and/or an external service like Weights and Biases^a).

^awandb.ai

6 Generating text

Now that we can train models, the last piece we need is the ability to generate text from our model.

Recall that a language model takes in a (possibly batched) integer sequence of length (`sequence_length`) and produces a matrix of size (`sequence_length` $\times$ `vocab_size`), where each element of the sequence is a probability distribution predicting the next word after that position. We will now write a few functions to turn this into a sampling scheme for new sequences.

Softmax By standard convention, the language model output is the output of the final linear layer (the “logits”) and so we have to turn this into a normalized probability via the softmax operation, which we saw earlier in Eq 6.

Decoding To generate text (decode) from our model, we will provide the model with a sequence of prefix tokens (the “prompt”), and ask it to produce a probability distribution over the vocabulary that predicts the next word in the sequence. Then, we will sample from this distribution over the vocabulary items to determine the next output token.

Concretely, one step of the decoding process should take in a sequence $x_{1...t}$ and return a token x_{t+1} via the following equation,

$$P(x_{t+1} = i | x_{1...t}) = \frac{\exp(v_i)}{\sum_j \exp(v_j)}$$

$$v = \text{TransformerLM}(x_{1...t})_t \in \mathbb{R}^{\text{vocab_size}}$$

where `TransformerLM` is our model which takes as input a sequence of `sequence_length` and produces a matrix of size (`sequence_length` $\times$ `vocab_size`), and we take the last element of this matrix, as we are looking for the next word prediction at the t -th position.

This gives us a basic decoder by repeatedly sampling from these one-step conditionals (appending our previously-generated output token to the input of the next decoding timestep) until we generate the end-of-sequence token `<|endoftext|>` (or a user-specified maximum number of tokens to generate).

Decoder tricks We will be experimenting with small models, and small models can sometimes generate very low quality texts. Two simple decoder tricks can help fix these issues. First, in temperature scaling we modify our softmax with a temperature parameter τ , where the new softmax is

$$\text{softmax}(v, \tau)_i = \frac{\exp(v_i/\tau)}{\sum_{j=1}^{|\text{vocab_size}|} \exp(v_j/\tau)}. \quad (15)$$

Insight: Temperature in Softmax

Varying τ can significantly affect the text generated by LLMs. We’d like you to consider why this is the case. First, describe what happens when you set $\tau = 0$, how does this affect the sampling process? Next, describe what happens τ is large.

Deliverable: 1-2 sentence explanation for each question.

Note how setting $\tau \rightarrow 0$ makes it so that the largest element of v dominates, and the output of the softmax becomes a one-hot vector concentrated at this maximal element.

Second, another trick is nucleus or top- p sampling, where we modify the sampling distribution by truncating low-probability words. Let q be a probability distribution that we get from a (temperature-scaled) softmax of size (`vocab_size`). Nucleus sampling with hyperparameter p

produces the next token according to the equation

$$P(x_{t+1} = i|q) = \begin{cases} \frac{q_i}{\sum_{j \in V(p)} q_j} & \text{if } i \in V(p) \\ 0 & \text{otherwise} \end{cases}$$

where $V(p)$ is the smallest set of indices such that $\sum_{j \in V(p)} q_j \geq p$. You can compute this quantity easily by first sorting the probability distribution q by magnitude, and selecting the largest vocabulary elements until you reach the target level of α .

Problem (decoding): Decoding

Deliverable: Implement a function to decode from your language model. We recommend that you support the following features:

- Generate completions for a user-provided prompt (i.e., take in some $x_{1..t}$ and sample a completion until you hit an `<|endoftext|>` token).
- Allow the user to control the maximum number of generated tokens.
- Given a desired temperature value, apply softmax temperature scaling to the predicted next-word distributions before sampling.
- Top- p sampling ([27]; also referred to as nucleus sampling), given a user-specified threshold value.

7 Experiments

Now it is time to put everything together and train (small) language models on a pretraining dataset.

7.1 How to Run Experiments and Deliverables

The best way to understand the rationale behind the architectural components of a Transformer is to actually modify it and run it yourself. There is no substitute for hands-on experience.

To this end, it's important to be able to **experiment quickly, consistently, and keep records of what you did**. To experiment quickly, we will be running experiments on a small scale model (17M parameters) and simple dataset (TinyStories). To do things consistently, you should log your experiments and learning curves associated with each experiment.

Problem (experiment_log): Experiment logging

For your training and evaluation code, create experiment tracking infrastructure that records configuration, training and validation loss, gradient steps, and wallclock time for each run.

Deliverable: Logging infrastructure code for your experiments, together with saved run logs and loss curves for the assignment problems below in this section.

7.2 Training

Hyperparameter tuning We will tell you some very basic hyperparameters to start with and ask you to find some settings for others that work well.

`vocab.size` 10000. Typical vocabulary sizes are in the tens to hundreds of thousands. You should vary this and see how the vocabulary and model behavior changes.

`context.length` 256. Simple datasets such as TinyStories might not need long sequence lengths.

`d_model` 512. This is slightly smaller than the 768 dimensions used in many small Transformer papers, but this will make things faster.

of layers and heads 4 layers, 8 heads. Together, this will give a fairly small Transformer.

total tokens processed 40,960,000 (your batch size $\times$ total step count $\times$ context length should equal roughly this value).

You should do some trial and error to find good defaults for the following other hyperparameters: learning rate, learning rate warmup, other AdamW hyperparameters (β_1 , β_2 , ϵ), and weight decay. You can find some typical choices of such hyperparameters in Kingma and Ba [22].

Putting it together Now you can put everything together by getting a trained BPE tokenizer, tokenizing the training dataset, and running this in the training loop that you wrote. **Important note:** For reference to a correct implementation, the above hyperparameters result in a roughly 1 hour runtime on Apple silicon (M1 Pro) for the TA solution. If you have runtimes that are much longer, please check and make sure your dataloading, checkpointing, or validation loss code is not bottlenecking your runtimes and that your implementation is properly batched. It will be helpful to set up simple profiling code to find where the bottlenecks may lie in your implementations.

Tips and tricks for debugging model architectures

- A common first step when developing any neural net architecture is to overfit to a single minibatch. If your implementation is correct, you should be able to quickly drive the training loss to near-zero. We **highly** recommend doing this. If you are unable to successfully overfit to a small subset of the dataset, it is extremely unlikely your model will train successfully on the larger set.
- Set debug breakpoints in various model components, and inspect the shapes of intermediate tensors to make sure they match your expectations.
- Monitor the norms of activations, model weights, and gradients to make sure they are not exploding or vanishing.

Problem (train): Train LM on TinyStories

Put it all together and train! Experiment with hyperparameters to get to a validation loss of 2.0. Make sure to save your best checkpoints alongside your logging and loss curves.

With your decoder in hand, we can now generate text! We will generate from the model and see how good it is.

Problem (generate): Generate text

Using your decoder and your trained checkpoint, report the text generated by your model. You may need to manipulate decoder parameters (temperature, top- p , etc.) to get fluent

outputs.

Deliverable: Text dump of at least 256 tokens of text (or until the first `<|endoftext|>` token), and a brief comment on the fluency of this output and at least two factors which affect how good or bad this output is.

7.3 Ablations and architecture modification

The best way to understand the Transformer is to actually modify it and see how it behaves. We will now do a few simple ablations and modifications.

Ablation 1: layer normalization It is often said that layer normalization is important for the stability of Transformer training. But perhaps we want to live dangerously. Let's remove LayerNorm from each of our Transformer blocks and see what happens.

Problem (layer_norm_ablation): Remove LayerNorm and train

Remove all of the LayerNorms from your Transformer and train. What happens with the same best-performing hyperparameters you found before? Can you get stability by using a lower learning rate?

Deliverable: A learning curve for when you remove LayerNorm and train, as well as a learning curve for the best learning rate.

Deliverable: A few sentence commentary on the impact of LayerNorm. Explain the "why" behind your observations.

Ablation 2: position embeddings We will next investigate the impact of the position embeddings on the performance of the model. Specifically, we will compare our base model (with Sinusoidal Positional Embeddings) with not including position embeddings at all (NoPE). It turns out that decoder-only transformers, i.e., those with a causal mask as we have implemented, can in theory infer relative or absolute position information without being provided with position embeddings explicitly [28, 29]. We will now test empirically how NoPE performs compared to Sinusoidal PE.

Problem (no_pos_emb): Implement NoPE

Modify your Transformer implementation to remove the position embedding information entirely and train, and see what happens.

Deliverable: A learning curve comparing the performance of Sinusoidal PE and NoPE.

Acknowledgements

This assignment was originally developed for Stanford's CS 336 course (cs336.stanford.edu) by Percy Liang and Tatsunori Hashimoto. We've adapted it for this course.

References

- [1] Ronen Eldan and Yuanzhi Li. TinyStories: How small can language models be and still speak coherent English?, 2023. arXiv:2305.07759.
- [2] Aaron Gokaslan, Vanya Cohen, Ellie Pavlick, and Stefanie Tellex. OpenWebText corpus. <http://Skylion007.github.io/OpenWebTextCorpus>, 2019.
- [3] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proc. of ACL*, 2016.
- [4] Changan Wang, Kyunghyun Cho, and Jiatao Gu. Neural machine translation with byte-level subwords, 2019. arXiv:1909.03341.
- [5] Philip Gage. A new algorithm for data compression. *C Users Journal*, 12(2):23–38, February 1994.
- [6] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners, 2019.
- [7] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training, 2018.
- [8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proc. of NeurIPS*, 2017.
- [9] Toan Q. Nguyen and Julian Salazar. Transformers without tears: Improving the normalization of self-attention. In *Proc. of IWSWLT*, 2019.
- [10] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the Transformer architecture. In *Proc. of ICML*, 2020.
- [11] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016. arXiv:1607.06450.
- [12] Hugo Touvron et al. Llama: Open and efficient foundation language models, 2023. arXiv:2302.13971.
- [13] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Proc. of NeurIPS*, 2019.
- [14] Aaron Grattafiori et al. The llama 3 herd of models, 2024. arXiv:2407.21783.
- [15] An Yang et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [16] Aakanksha Chowdhery et al. PaLM: Scaling language modeling with pathways, 2022. arXiv:2204.02311.
- [17] Dan Hendrycks and Kevin Gimpel. Bridging nonlinearities and stochastic regularizers with Gaussian error linear units, 2016. arXiv:1606.08415.
- [18] Stefan Elfving, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning, 2017.

- [19] Yann N. Dauphin, Angela Fan, Michael Auli, and David Grangier. Language modeling with gated convolutional networks, 2017.
- [20] Noam Shazeer. GLU variants improve transformer, 2020. arXiv:2002.05202.
- [21] Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding, 2021.
- [22] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proc. of ICLR*, 2015.
- [23] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *Proc. of ICLR*, 2019.
- [24] Tom B. Brown et al. Language models are few-shot learners. In *Proc. of NeurIPS*, 2020.
- [25] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020. arXiv:2001.08361.
- [26] Jordan Hoffmann et al. Training compute-optimal large language models, 2022. arXiv:2203.15556.
- [27] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *Proc. of ICLR*, 2020.
- [28] Yao-Hung Hubert Tsai, Shaojie Bai, Makoto Yamada, Louis-Philippe Morency, and Ruslan Salakhutdinov. Transformer dissection: An unified understanding for transformer’s attention via the lens of kernel. In *Proc. of EMNLP-IJCNLP*, 2019.
- [29] Amirhossein Kazemnejad, Inkit Padhi, Karthikeyan Natesan, Payel Das, and Siva Reddy. The impact of positional encoding on length generalization in transformers. In *NeurIPS*, 2023.